

ExactGED: ILP Benchmarking for Exact GED and Optimal Solutions Dataset

Andrea D’Ascenzo¹ (✉)^[0000–0001–5612–0798], Julian Meffert²^[0009–0008–9670–4569], Petra Mutzel²^[0000–0001–7621–971X], and Fabrizio Rossi³^[0000–0002–7495–390X].

¹ Gran Sasso Science Institute, L’Aquila, Italy andrea.dascenzo@gssi.it

² University of Bonn, Bonn, Germany s6jumeff@uni-bonn.de, pmutzel@uni-bonn.de

³ University of L’Aquila, L’Aquila, Italy fabrizio.rossi@univaq.it

Abstract. The Graph Edit Distance (GED) is one of the most widely used measures of graph similarity in practice. Given two labelled graphs G and H , the GED is defined as the minimum-cost sequence of edit operations required to transform G into H , where the allowed operations include vertex and edge insertions and deletions, as well as label substitutions. Computing the GED is NP-hard. In practice, state-of-the-art algorithms rely on either Integer Linear Programming (ILP) formulations or A^* -search-based methods. Heuristic approaches based on machine learning have also been proposed to approximate the optimal GED.

In this paper, we provide an extensive computational study of exact methods for GED and demonstrate that the size of optimally solvable instances has increased dramatically due to the recent introduction of stronger ILP formulations. A preliminary comparison with the A^* -based algorithms indicates that they are outperformed by the ILP-based approaches on graphs exceeding ten nodes. Additionally, we release a dataset of optimal edit paths across widely used datasets from the TUDataset, which can serve as a robust, exact knowledge base for both interpretable and non-interpretable machine learning methods. This dataset will be continuously updated as new results are computed.

Keywords: Graph Edit Distance · Graph Similarity · Integer Linear Programming · A^* -search.

1 Introduction

Graphs are a fundamental abstraction for representing structured data and arise naturally in a wide range of application domains, including chemistry, bioinformatics, computer vision, social networks, and pattern recognition. A central task in many of these applications is the comparison of graphs in order to quantify their structural similarity. Among the numerous similarity measures proposed in the literature, the *Graph Edit Distance* (GED) is one of the most widely used.

Given two labelled graphs G and H , the GED is defined as the minimum-cost sequence of edit operations required to transform G into H . Typical edit

operations include vertex and edge insertions, deletions, and label substitutions. GED has found applications in molecular similarity analysis, image recognition, error-tolerant pattern matching, and graph-based machine learning.

Despite its versatility, computing the exact GED is computationally challenging. The problem is known to be NP-hard, and its exact computation rapidly becomes prohibitive as graph sizes increase. Consequently, a significant body of research has focused on the design of efficient exact and approximate algorithms. Comprehensive surveys on exact and approximate algorithms for GED are provided in [9,16,3]. Among exact approaches, Riesen et al. [17] proposed an application of the classical A^* search algorithm to exact GED computation. Their method explores a search tree in which each node represents a partial edit path, i.e., a sequence of edit operations applied so far, while complete edit paths correspond to leaves of the tree. The exploration is guided by heuristic estimations of the remaining edit cost, allowing the algorithm to prioritize the most promising search branches. In practice, however, this approach suffers from the extremely large search space that must be explored, resulting in prohibitive time and memory requirements [13,4]. Several improvements to the A^* -search paradigm have been proposed in recent years. For example, Chen et al. [7] combined A^* search with the beam-stack-search paradigm, while Kim et al. [12] exploited graph isomorphisms to avoid redundant computations. Nevertheless, experimental evaluations of these methods are generally limited to relatively small graphs, typically containing at most 30 vertices.

More recently, Chang et al. [5,6] introduced two improved A^* -based algorithms, namely ASTAR-LSA and ASTAR-BMAO. By combining stronger lower-bound estimations with specialized data structures for compactly storing graph node labels, these methods significantly extended the range of tractable exact GED instances. In particular, they were able to process instances involving graphs with up to 64 vertices, establishing new state-of-the-art results for exact GED computation. Note that A^* -based methods also form the foundation of several heuristic-guided approaches for approximate GED computation [2,10].

On the other hand, Integer Linear Programming (ILP) techniques are regarded as some of the most effective approaches for the exact computation of the GED (see, e.g., [1,13,4]). Justice and Hero [11] proposed one of the earliest ILP formulations for GED, achieving strong practical performance. However, their formulation is limited to graphs with no edge labels.

To address this limitation, Lerouge et al. [13] introduced two ILP formulations, referred to as F1 and F2, both capable of handling edge-labelled graphs. Among these, formulation F2 has consistently demonstrated superior practical performance [4]. As a consequence, it has become a standard component in several pattern recognition applications, including recent work by Xu et al. [18].

Until recently, even ILP formulations struggled to solve GED instances on graphs exceeding 50 vertices. D’Ascenzo et al. [8] proposed enhancements to both the F1 and F2 formulations by introducing valid inequalities that significantly strengthen the models. In addition, they presented a new formulation, referred to as FORI, which is shown to be both theoretically and practically stronger

than all previously known approaches, enabling the solution of GED instances with up to 100 vertices and edges per graph.

In this work, we provide a computational comparison of exact methods for GED. Our study evaluates the practical performance of exact algorithms across graph families from the TU dataset collection. In addition, we release a dataset containing optimal edit paths for these graph families, which can serve as a knowledge base for supervised machine learning methods. Both the formulation implementations as well as the optimum edit paths are available at <https://github.com/D-hash/ExactGED>. This dataset will be continuously expanded as new optimal solutions are computed.

2 Notation

We follow the notation used in [6]. Let $G = (V_G, E_G, l)$ be a simple, undirected, labelled⁴ graph, where V_G is the set of nodes, E_G is the set of edges, and $l : V_G \cup E_G \rightarrow \Sigma_V \cup \Sigma_E$ is a labelling function that assigns to each vertex $u \in V_G$ a label $l(u) \in \Sigma_V$ and to each edge $\{u, v\} \in E_G$ a label $l(u, v) \in \Sigma_E$.

A node $u \in V_G$ is considered a neighbour (or adjacent) to a node $v \in V_G$ if an edge $\{u, v\} \in E_G$ exists. For convenience, we may use the shorthand notation uv in place of $\{u, v\}$. The set of edges incident to a node u in G , is denoted as $\delta_G(u) = \{\{u, v\} \in E_G \mid v \in V_G\}$, the neighbourhood of u in G is denoted as $\Gamma_G(u) = \{v \in V_G \mid \{u, v\} \in E_G\}$.

To accommodate node and edge insertions or deletions, we extend the graph with a dummy node ε_V and a dummy edge ε_E , resulting in the augmented sets $V_{G+\varepsilon} = V_G \cup \{\varepsilon_V\}$ and $E_{G+\varepsilon} = E_G \cup \{\varepsilon_E\}$. The specific labels and associated cost functions for nodes and edges depend on the dataset and application context.

Although most datasets in the literature consist of undirected graphs, the definitions and notation presented here can be readily adapted to directed graphs. In such cases, a directed edge from node u to node v is denoted by (u, v) , with $\delta_G^+(u)$ and $\delta_G^-(u)$ representing the sets of outgoing and incoming edges of u , respectively.

The *graph edit distance* (GED) between two graphs G and H , denoted as $GED(G, H)$, represents the minimum number of edit operations required to transform graph G into graph H . These edit operations include: (1) inserting a labelled vertex; (2) deleting a labelled vertex; (3) changing the label of a vertex; (4) inserting a labelled edge; (5) deleting a labelled edge; and (6) modifying the label of an edge. Each edit operation is associated with a non-negative *edit cost*: $c_V : \Sigma_V \times \Sigma_V \rightarrow \mathbb{R}_{\geq 0}$ for node operations, $c_E : \Sigma_E \times \Sigma_E \rightarrow \mathbb{R}_{\geq 0}$ for edge operations.

⁴ Throughout this paper, we use the term *label* to refer to node or edge annotations, which are also commonly known as attributes in other domains.

3 State of the Art

This section provides an overview of the state-of-the-art exact approaches for the GED problem. This consists of the ILP formulations FORI, F2+ and F1+ presented in [8] and the A^* -based approaches ASTAR-LSA [5] and ASTAR-BMAO [6].

3.1 Integer Linear Programming Formulations

The first formulation, F1+, see Figure 1, is a direct model that uses assignment variables to represent node and edge correspondences. It includes two classes of decision variables. For nodes, $x_{i,k} = 1$ indicates that node $i \in V_G$ is matched to node $k \in V_H$, whereas $x_{i,\varepsilon} = 1$ and $x_{\varepsilon,k} = 1$ represent the deletion of node i and the insertion of node k , respectively. Analogously, for edges, $y_{ij,kl} = 1$, $y_{ij,\varepsilon} = 1$, and $y_{\varepsilon,kl} = 1$ indicate the matching, deletion, and insertion of edges $\{i, j\} \in E_G$ and $\{k, l\} \in E_H$. The formulation contains $xcard_{F1} = |V_G||V_H| + |V_G| + |V_H|$ node assignment variables and $ycard_{F1} = |E_G||E_H| + |E_G| + |E_H|$ edge assignment variables. The objective function directly captures the total cost associated with the edit operations induced by the selected variables.

Constraints (1a) ensure that each node of graph G is either matched to a node of graph H or deleted. Likewise, constraints (1b) guarantee that every node of H is either the image of a node in G or is inserted. Constraints (1c) and (1d) impose the analogous requirements on edges, ensuring that each edge is either matched to an edge in the other graph, deleted, or inserted. The topological constraints (1e) and (1f) establish the consistency between node and edge assignments. In particular, they ensure that an edge $\{i, j\} \in E_G$ can be matched to an edge $\{k, l\} \in E_H$ only if the endpoints i and j are consistently mapped to k and l , respectively, in either order. Whenever this condition is violated, the corresponding edge assignment variable $y_{ij,kl}$ is forced to take value zero.

Lerouge et al. [13] observed that the vertex and edge insertion/deletion variables are redundant, as their values are uniquely determined by the remaining assignment variables. Consequently, these variables can be eliminated from the formulation, with the corresponding insertion and deletion costs incorporated directly into the objective function as follows.

The binary variable $x_{i,k}$ takes value 1 if node $i \in V_G$ is matched to node $k \in V_H$, and 0 otherwise. Likewise, the binary variable $y_{ij,kl}$ is equal to 1 when edge $\{i, j\} \in E_G$ is matched to edge $\{k, l\} \in E_H$, and 0 otherwise.

The constant term

$$K = \sum_{i \in V_G} c_{i,\varepsilon} + \sum_{k \in V_H} c_{\varepsilon,k} + \sum_{ij \in E_G} c_{ij,\varepsilon} + \sum_{kl \in E_H} c_{\varepsilon,kl}$$

represents the total cost of deleting all nodes and edges of G and inserting all nodes and edges of H . It is added to the objective function, while the assignment variables account for the cost reductions associated with matched nodes and edges. The substitution costs are defined as

$$\bar{c}_{i,k} = c_{i,k} - c_{i,\varepsilon} - c_{\varepsilon,k}, \quad \forall (i, k) \in V_G \times V_H,$$

$$\begin{aligned}
\min \quad & \sum_{i \in V_G} \sum_{k \in V_H} c_{i,k} x_{i,k} + \sum_{i \in V_G} c_{i,\varepsilon} x_{i,\varepsilon} + \sum_{k \in V_H} c_{\varepsilon,k} x_{\varepsilon,k} \\
& + \sum_{ij \in E_G} \sum_{kl \in E_H} c_{ij,kl} y_{ij,kl} + \sum_{ij \in E_G} c_{ij,\varepsilon} y_{ij,\varepsilon} + \sum_{kl \in E_H} c_{\varepsilon,kl} y_{\varepsilon,kl} \\
\text{s.t.} \quad & x_{i,\varepsilon} + \sum_{k \in V_H} x_{i,k} = 1 \quad \forall i \in V_G \quad (1a) \\
& x_{\varepsilon,k} + \sum_{i \in V_G} x_{i,k} = 1 \quad \forall k \in V_H \quad (1b) \\
& y_{ij,\varepsilon} + \sum_{kl \in E_H} y_{ij,kl} = 1 \quad \forall ij \in E_G \quad (1c) \\
& y_{\varepsilon,kl} + \sum_{ij \in E_G} y_{ij,kl} = 1 \quad \forall kl \in E_H \quad (1d) \\
& \sum_{kl \in \delta_H(k)} y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall k \in V_H, ij \in E_G \quad (1e) \\
& \sum_{ij \in \delta_G(i)} y_{ij,kl} \leq x_{i,k} + x_{i,l} \quad \forall i \in V_G, kl \in E_H \quad (1f) \\
& x \in \{0, 1\}^{x_{card_{F1+}}} \quad (1g) \\
& y \in \{0, 1\}^{y_{card_{F1+}}} \quad (1h)
\end{aligned}$$

Fig. 1. Formulation F1+ proposed in [8].

for node pairs, and

$$\bar{c}_{ij,kl} = c_{ij,kl} - c_{ij,\varepsilon} - c_{\varepsilon,kl}, \quad \forall (ij, kl) \in E_G \times E_H,$$

for edge pairs. Consequently, whenever $x_{i,k} = 1$, the corresponding coefficient in the objective function replaces the cost of deleting node $i \in V_G$ and inserting node $k \in V_H$ with the cost of mapping i to k . The same reasoning applies to the edge assignment variables: if $y_{ij,kl} = 1$, the deletion and insertion costs associated with edges ij and kl are replaced by their substitution cost. In these terms, the formulation F2+ of [8] is shown in Figure 2.

A major advance in ILP formulations for the GED problem is represented by FORI [8]. The key idea underlying this formulation is the introduction of suitable edge orientations for the input graphs. The node assignment variables x are the same as those used in F2+ and define a node mapping between the two graphs. To model edge correspondences, each undirected edge $\{i, j\} \in E_G$ is oriented from the smaller-indexed endpoint to the larger one, i.e., $i < j$, yielding the directed graph $\vec{G}$. For graph H , each undirected edge $\{k, l\} \in E_H$ with $k \neq l$ is replaced by the two arcs (k, l) and (l, k) , resulting in the bidirected graph $\overleftrightarrow{H}$. Based on these orientations, FORI introduces binary variables $z_{ij,kl}$ for every

$$\begin{aligned}
& \min \sum_{i \in V_G} \sum_{k \in V_H} \bar{c}_{i,k} \cdot x_{i,k} + \sum_{ij \in E_G} \sum_{kl \in E_H} \bar{c}_{ij,kl} \cdot y_{ij,kl} + K \\
& \text{s.t.} \quad \sum_{k \in V_H} x_{i,k} \leq 1 \quad \forall i \in V_G \quad (2a) \\
& \quad \sum_{i \in V_G} x_{i,k} \leq 1 \quad \forall k \in V_H \quad (2b) \\
& \quad \sum_{kl \in \delta_H(k)} y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall k \in V_H, ij \in E_G \quad (2c) \\
& \quad \sum_{ij \in \delta_G(i)} y_{ij,kl} \leq x_{i,k} + x_{i,l} \quad \forall i \in V_G, kl \in E_H \quad (2d) \\
& \quad x \in \{0, 1\}^{|V_G| \cdot |V_H|} \quad (2e) \\
& \quad y \in \{0, 1\}^{|E_G| \cdot |E_H|} \quad (2f)
\end{aligned}$$

Fig. 2. Formulation F2+ proposed in [8].

arc $(i, j) \in E_{\vec{G}}$ and every arc $(k, l) \in E_{\vec{H}}$. A variable $z_{ij,kl}$ takes value 1 if arc (i, j) is matched to arc (k, l) , and 0 otherwise.

The objective function of FORI is inherited from F2+. In particular, the variables $z_{ij,kl}$ and $z_{ij,lk}$ replace the corresponding edge assignment variable $y_{ij,kl}$ of F2+ and carry the same objective coefficients. The complete formulation FORI is reported in Figure 3.

The introduction of variables that explicitly represent directed edge mappings allows the relationships between corresponding nodes to be modeled more accurately. In particular, FORI distinguishes whether the arc (i, j) , with $i < j$, in $\vec{G}$ is mapped to arc (k, l) or to arc (l, k) in $\vec{H}$. As a result, the formulation captures which of the nodes $i, j \in V_G$ is matched to node $k \in V_H$ and which is matched to node $l \in V_H$. This additional information makes it possible to strengthen the formulation by decomposing the original constraint (2c) into two separate families of constraints, each involving only a single node assignment variable on the right-hand side; see constraints (3c) and (3d).

3.2 A*-based Approaches

A*-based algorithms compute the GED by continually expanding partial node maps from V_G to V_H , where $GED(G, H)$ finally equals the minimum edit cost among all complete node maps. This results in a search tree, with nodes at level i corresponding to an injective (partial) mapping $f : \{v_1, \dots, v_i\} \hookrightarrow V_H$, in which A*-based algorithms perform a pruned best-first search, exploiting lower bounds for prioritization and pruning.

Let $L_V(X)$ and $L_E(Y)$ be the multi-set of vertex and resp. edge labels of $X \subseteq V_G$ and $Y \subseteq E_G$ and f be a partial node map. We denote the subgraphs

$$\begin{aligned}
\min \quad & \sum_{i \in V_G} \sum_{k \in V_H} \bar{c}_{i,k} \cdot x_{i,k} + \sum_{(i,j) \in E_{\vec{G}}} \sum_{(k,l) \in E_{\vec{H}}} \bar{c}_{ij,kl} \cdot z_{ij,kl} + K \\
\text{s.t.} \quad & \sum_{k \in V_H} x_{i,k} \leq 1 \quad \forall i \in V_G \quad (3a) \\
& \sum_{i \in V_G} x_{i,k} \leq 1 \quad \forall k \in V_H \quad (3b) \\
& \sum_{kl \in \delta_{\vec{H}}^+(k)} z_{ij,kl} \leq x_{i,k} \quad \forall k \in V_H, (i,j) \in E_{\vec{G}} \quad (3c) \\
& \sum_{lk \in \delta_{\vec{H}}^-(k)} z_{ij,lk} \leq x_{j,k} \quad \forall k \in V_H, (i,j) \in E_{\vec{G}} \quad (3d) \\
& \sum_{ij \in \delta_{\vec{G}}^+(i)} z_{ij,kl} + \sum_{ji \in \delta_{\vec{G}}^-(i)} z_{ji,lk} \leq x_{i,k} \quad \forall i \in V_G, (k,l) \in E_{\vec{H}} \quad (3e) \\
& x \in \{0,1\}^{|V_G| \cdot |V_H|} \quad (3f) \\
& z \in \{0,1\}^{|E_G| \cdot 2|E_H|} \quad (3g)
\end{aligned}$$

Fig. 3. Formulation FORI proposed in [8].

induced by the mapped nodes by $G[f]$, $H[f]$ and the subgraphs induced by the unmapped nodes by $G[\bar{f}]$, $H[\bar{f}]$ respectively. We denote by $L_C(v)$ the multi-set of labels of v 's incident edges crossing between mapped and unmapped nodes and by $L_{E_I}(v)$ the multi-set of labels of v 's incident inner edges running between unmapped vertices.

Definition 1 (Anchor-aware label set-based lower bound [5]). *For two graphs G, H , the anchor-aware label set-based lower bound (LSA) is defined as*

$$\begin{aligned}
LSA(f) &= COST(f) + LB_{LSA}(f) \\
LB_{LSA}(f) &= \Upsilon(L_V(V_{G[\bar{f}]}), L_V(V_{H[\bar{f}]}) + \Upsilon(L_E(E_{G[\bar{f}]}), L_E(E_{H[\bar{f}]}) \\
&\quad + \sum_{v \in V(G[f])} \Upsilon(L_C(v), L_C(f(v))).
\end{aligned}$$

where $\Upsilon(S_1, S_2) = \max\{|S_1|, |S_2|\} - |S_1 \cap S_2|$ denotes the edit distance between two multi-sets S_1, S_2 .

The LSA lower bound can be interpreted as adding the number of vertex and edge label mismatches between the unmatched vertices, unmatched inner edges and unmatched crossing edges, taking into account the already mapped nodes (also called *anchored* nodes). Thus, LSA can only be computed for graphs with categorical labels and unit edit costs.

The ASTAR-LSA algorithm employs the LSA lower bound for the best-first tree search.

Definition 2. *The revised branch structure of a vertex $v \in G[\bar{f}]$ is defined as*

$$B_f(v) = (l(v), L_{E_I}(v), \bigcup_{w \in V(G[f])} \{(f(w), l(v, w))\}),$$

where $l(v, w) = \perp$ if there is no edge between v and w .

Based on the branch structures $B_f(v)$ of $v \in G[\bar{f}]$ and $B(u)$ of $u \in H[\bar{f}]$ the cost of mapping v to u is defined as

$$\begin{aligned} \lambda_f^{\text{BMA}}(v, u) &= \delta_{l(v) \neq l(u)} + \frac{1}{2} \mathcal{T}(L_{E_I}(v), L_{E_I}(u)) \\ &+ \sum_{w \in V(G[f])} \delta_{l(v, w) \neq l(u, f(w))}, \end{aligned}$$

where $\mathcal{T}(\cdot, \cdot)$ is multiplied by a factor of $\frac{1}{2}$ because the edge $(v, w) \in E_G$ can produce costs both in the matching of $B_f(v)$ and $B_f(w)$ and δ_{cond} is the Dirac delta with $\delta_{\text{cond}} = 1$ if cond evaluates as true and $\delta_{\text{cond}} = 0$ otherwise. This value can be interpreted as the minimum cost to edit v and its adjacent edges to be the same as u ’s, while fixing the already mapped nodes and their induced edge mappings.

Let $\Pi(G, H)$ denote the set of full mappings between the vertices $V_{G+\varepsilon}$ and $V_{H+\varepsilon}$.

Definition 3 (Anchor-aware branch match-based lower bound [6]). *The anchor-aware branch match-based lower bound BMA of two graphs G, H is defined as*

$$\begin{aligned} \text{BMA}(f) &= \text{COST}(f) + \text{LB}_{\text{BMA}}(f) \\ \text{LB}_{\text{BMA}}(f) &= \min_{\sigma \in \Pi(G[\bar{f}], H[\bar{f}])} \sum_{v \in V(G[\bar{f}])} \lambda_f^{\text{BMA}}(v, \sigma(v)) \end{aligned}$$

Let $f : \{v_1, \dots, v_i\} \hookrightarrow V_H$ be a partial node map. In children of f , which we denote by $f \oplus \{v_{i+1} \mapsto u \in V_{H[\bar{f}]}\}$, the branch structures of the nodes of $G[\bar{f}]$ and $H[\bar{f}]$ are the same for different choices of u . Thus, computations can be shared between calculations of $\text{LB}_{\text{BMA}}(f \oplus \{v_{i+1} \mapsto u\})$ and $\text{LB}_{\text{BMA}}(f \oplus \{v_{i+1} \mapsto u'\})$. Together with efficient data structures, ASTAR-BMAO uses this optimized version of BMA, called BMAO as the lower bound heuristic for the best-first tree search.

4 Experiments

In the experimental comparison provided by [8] the authors restricted to compare the newly introduced formulations F2+ and FORI against the previous state-of-the-art formulation, namely F2 [13]. Moreover, the dataset employed

where restricted to three types, drawn from the same area of application such as bioinformatics. Here we extend the experimental comparison to compare the three formulations F1+, F2+, and FORI with the state-of-the-art combinatorial algorithms ASTAR-LSA [5] and ASTAR-BMAO [6], we use datasets from the *TUDataset* repository [15]. This collection comprises multiple labeled graph datasets drawn from diverse application domains. Table 1 summarizes the considered datasets and their main characteristics.

Table 1. Dataset statistics and graph properties [15].

Dataset ($\mathcal{D}$)	$ \mathcal{D} $	Classes	$ V $	$ E $	$ \Sigma_V $	$ \Sigma_E $
AIDS	2 000	2	15.7	16.2	✓	✓
ALCHEMY	202 579	12	10.1	10.4	✓	✓
ENZYMES	600	6	32.6	62.1	✓	✗
IMDB-BINARY	1 000	2	19.8	96.5	✗	✗
NCI1	4 110	2	29.9	32.3	✓	✗
PTC-FM	349	2	14.1	14.5	✓	✗
REDDIT-BINARY	2 000	2	429.6	497.8	✗	✗
REDDIT-THREADS	203 088	2	23.9	24.9	✗	✗
YEAST	79 601	2	21.5	22.8	✓	✓
ZINC	249 456	12	23.1	24.9	✓	✓

We implemented the ILP formulations in Python and solved them using Gurobi 13. For each instance, we considered both linear relaxation and the corresponding integer program. We set the primal-dual hybrid gradient (Gurobi parameter *Method* set to value 6) as LP solver, which has proven to outperform simplex and barrier algorithms in a preliminary experience. All other Gurobi parameters are at the default settings. On the other hand, for ASTAR-LSA we used the implementation provided by the authors at https://github.com/LijunChang/Graph_Edit_Distance. The experiments were run on a workstation with AMD EPYC 7282 32-core 3.2GHz CPU, 504GB RAM, Ubuntu 20.04.6. We impose a time limit of ten minutes per graph pair and set the memory limit to 50GB to reflect typical workstation capabilities.

We analyse the performance of the three formulations across the considered datasets. Table 2 reports the statistics for F1+, F2+, FORI, ASTAR-LSA [5] and ASTAR-BMAO [6]. For each dataset $\mathcal{D}$, we report $|\overline{\mathcal{D}}|$, the number of evaluated graph pairs; #, the number of instances solved within the time limit. For the ILP formulations we additionally report TIME, the average time required to solve the integer program; GAP, the average optimality gap at termination; NODES, the average number of branch-and-bound nodes explored; LR-TIME, the average time to solve the linear relaxation; and ROOTGAP, the optimality gap of the root relaxation.

Overall, the results show a clear dependency between instance difficulty and graph size, particularly in terms of the average number of nodes $|V|$ and edges

Table 2. Algorithm statistics on all the tested datasets. Marked numbers of solved instances * containing instances that reached the time limit appeared to stall within certain Gurobi routines. Changing the random seed allowed the solver to complete these instances within average running times. For insights, the interested reader is referred to the *performance variability* phenomenon [14].

Dataset	$\overline{\mathcal{P}}$	F1+						F2+						FORI						ASTAR-LSA		ASTAR-BMAO	
		#	TIME	GAP	NODES	LR-TIME	ROOTCAP	#	TIME	GAP	NODES	LR-TIME	ROOTCAP	#	TIME	GAP	NODES	LR-TIME	ROOTCAP	#	TIME	#	TIME
AIDS	7357	7357	0.12	0	2.3	0.02	2.6	7357	0.1	0	3.4	0.01	2.6	7357	0.01	0	1.0	< 0.01	1.1	7302	5.3	7298	6.3
ALC	10000	10000	0.01	0	1.0	< 0.01	5.2	10000	0.01	0	1.0	< 0.01	5.2	10000	0.01	0	1.0	< 0.01	2.3	10000	< 0.01	10000	< 0.01
ENZ	100	48	363.6	0.1	2813.5	2.4	20.7	50	351.1	0.1	4031.7	0.4	20.7	59	310.1	< 0.1	467.2	0.7	12.7	29	443.0	35	412.2
IMDB-B	10000	9479	65.2	< 0.1	40.2	6.7	6.2	9570	54.2	4.7	106.3	3.9	6.2	9621	54.4	9.1	295.7	6.2	5.0	6592	217.9	7947	141.2
NCI1	700	693	18.9	< 0.1	130.2	0.4	5.4	689	23.2	< 0.1	263.4	0.1	5.4	699*	2.6	0.3	6.8	0.1	1.8	450	233.6	448	243.4
PTC-FM	34900	34895	0.2	< 0.1	2.6	0.01	3.1	34892	0.3	< 0.1	3.5	< 0.01	3.1	34896*	0.1	< 0.1	1.1	< 0.01	0.9	34493	8.8	34733	4.5
RED-B	110	61	296.9	0.1	32.7	101.2	1.4	70	257.6	8.8	53.0	40.9	1.5	96	97.3	16.8	2.6	19.7	0.4	17	512.3	16	516.3
RED-T	10000	9998	0.4	< 0.1	2.1	0.04	0.8	9998	0.2	< 0.1	1.0	0.02	0.8	10000	0.04	0	1.0	0.01	0.3	5686	268.7	7513	160.4
YEAST	10000	9997	0.7	< 0.1	10.1	0.1	2.7	9997	0.6	< 0.1	18.2	0.04	2.7	9998*	0.2	< 0.1	1.4	0.02	1.0	8839	84.4	9232	60.9
ZINC	10000	10000	0.3	0	2.7	0.1	3.6	10000	0.2	0	2.8	0.05	3.6	9998*	0.2	< 0.1	1.1	0.02	1.8	8965	94.5	9691	41.7

$|E|$. Datasets with small and sparse graphs (e.g., AIDS, ALC, PTC-FM, RED-T, YEAST, and ZINC) are solved almost entirely to optimality by all formulations, with negligible running times and negligible positive optimality gaps. In contrast, datasets with larger or denser graphs (e.g., RED-B, IMDB-B, and ENZ) exhibit significantly higher computational effort, both in terms of runtime and branch-and-bound exploration. In particular, datasets with larger average graph size, such as RED-B ($|V| \approx 430$) and ENZ ($|V| \approx 32$ but with high edge density), consistently require orders of magnitude more search nodes compared to smaller datasets such as PTC-FM or AIDS. This suggests that the combinatorial explosion in the GED search space is more sensitive to structural complexity (edge density) than to node count alone.

Among the three ILP formulations, FORI consistently achieves the best performance in terms of number of solved instances and runtime, especially on moderately sized graphs. It significantly reduces both NODES and TIME compared to F1+ and F2+, indicating a tighter formulation that leads to stronger pruning in the branch-and-bound process. This effect is particularly evident on ENZ and RED-B, where FORI reduces the search tree size by up to an order of magnitude.

The behaviour of the linear relaxation further explains these differences. Formulations with lower ROOTGAP generally lead to fewer explored nodes and faster convergence. In this regard, FORI exhibits substantially smaller root gaps (e.g., on RED-B and IMDB-B), which correlates with improved solver efficiency. Conversely, larger root gaps in F1+ and F2+ translate into weaker relaxations and increased branching effort.

Finally, we observe that runtime improvements are not always proportional to reductions in NODES. For example, on RED-B, F2+ explores more nodes than FORI but achieves competitive runtimes due to differences in node processing cost. This highlights that both formulation strength and per-node computational complexity jointly determine overall performance.

In summary, the experimental results indicate a strong correlation between graph structural properties (notably size and density), the quality of the linear relaxation (ROOTGAP), and solver performance. Among the tested formulations, FORI provides the most robust behaviour across heterogeneous datasets.

Turning to ASTAR-LSA and ASTAR-BMAO, both methods perform competitively on very small graphs, such as those in the ALC dataset, which on average contain fewer than 11 vertices and edges, solving each GED instance faster than the three ILP formulations.

For every dataset except ALC, at least one of the ILP formulations solves more instances than both combinatorial approaches. The differences are especially large on the more challenging benchmarks. On RED-B, for example, FORI solves 96 out of 110 instances, whereas ASTAR-LSA and ASTAR-BMAO solve only 17 and 16 instances, respectively. Similarly, on NCI1 the best ILP formulation solves 699 instances, compared with 450 and 448 for ASTAR-LSA and ASTAR-BMAO. On RED-T, where all ILP formulations solve essentially every instance, the combinatorial algorithms solve only 5686 and 7513 instances out of 10000. Comparable trends are observed on YEAST and ZINC, where the ILP formulations

solve all instances while the combinatorial methods fail on a substantial fraction of the benchmark.

Comparing the two combinatorial algorithms, ASTAR-BMAO generally improves upon ASTAR-LSA in terms of the number of solved instances and average runtime on several datasets, including IMDB-B, PTC-FM, RED-T, YEAST, and ZINC. Nevertheless, the improvements remain insufficient to close the gap with the ILP-based approaches.

Overall, the formulation FORI emerges as the most effective ILP model, consistently providing the highest solution rate and strongest relaxations. More importantly, the results show that ILP formulations substantially outperform current state-of-the-art A^* -based exact algorithms on a broad range of graph classes, often by large margins both in terms of solved instances and computational time.

5 Conclusion

In this work, we provided an extensive computational study on exact graph edit distance computation across widely used datasets from the *TUDataset*, alongside a newly introduced dataset of optimum edit paths. Our benchmark explored state-of-the-art algorithms from both the class of ILP-based and A^* -based approaches. Among the methods considered, the FORI ILP formulation consistently achieved the best performance, outperforming F1+ and F2+ across all evaluated metrics. In contrast, the A^* -based algorithms ASTAR-BMAO and ASTAR-LSA proved to be efficient only on very small graphs with around ten nodes. The ILP-based approaches also offer superior flexibility in terms of the ability to support more general edit cost functions through a simple change of the objective coefficients, whereas current state-of-the-art A^* -based approaches remain limited to unit edit costs. Ultimately, our results demonstrate that the size of optimally solvable instances has increased dramatically due to the recent introduction of these stronger polyhedral formulations. This opens immediate practical possibilities for the graph machine learning community. While researchers have historically been forced to rely on heuristic approximations due to the combinatorial explosion of previous exact solvers, optimal GED values are now computationally accessible. Our dataset enables their direct integration into supervised learning pipelines and provides a robust, exact knowledge base for training interpretable and non-interpretable models. Future work should focus on a collaborative, community driven effort to continually expand these solved benchmarks. Scaling these datasets will simultaneously provide high-quality training data for learning-based approaches and offer the combinatorial optimization community a challenging testing ground to further develop A^* -based algorithms and ILP-based approaches.

Acknowledgments. Andrea D’Ascenzo is partially supported by the MUR (Italy) Department of Excellence 2023–2027.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Abu-Aisheh, Z., Gauzere, B., Bougleux, S., Ramel, J.Y., Brun, L., Raveaux, R., Heroux, P., Adam, S.: Graph edit distance contest: Results and future challenges. *Pattern Recognition Letters* **100**, 96–103 (2017). <https://doi.org/https://doi.org/10.1016/j.patrec.2017.10.007>, <https://www.sciencedirect.com/science/article/pii/S0167865517303690>
2. Abu-Aisheh, Z., Raveaux, R., Ramel, J., Martineau, P.: An exact graph edit distance algorithm for solving pattern recognition problems. In: Marsico, M.D., Figueiredo, M.A.T., Fred, A.L.N. (eds.) *ICPRAM 2015 - Proceedings of the International Conference on Pattern Recognition Applications and Methods*, Volume 1, Lisbon, Portugal, 10–12 January, 2015. pp. 271–278. SciTePress (2015)
3. Blumenthal, D.B., Boria, N., Gamper, J., Bougleux, S., Brun, L.: Comparing heuristics for graph edit distance computation. *VLDB J.* **29**(1), 419–458 (2020). <https://doi.org/10.1007/S00778-019-00544-1>, <https://doi.org/10.1007/s00778-019-00544-1>
4. Blumenthal, D.B., Gamper, J.: On the exact computation of the graph edit distance. *Pattern Recognit. Lett.* **134**, 46–57 (2020). <https://doi.org/10.1016/J.PATREC.2018.05.002>
5. Chang, L., Feng, X., Lin, X., Qin, L., Zhang, W., Ouyang, D.: Speeding up GED verification for graph similarity search. In: *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20–24, 2020*. pp. 793–804. IEEE (2020). <https://doi.org/10.1109/ICDE48307.2020.00074>, <https://doi.org/10.1109/ICDE48307.2020.00074>
6. Chang, L., Feng, X., Yao, K., Qin, L., Zhang, W.: Accelerating graph similarity search via efficient GED computation. *IEEE Trans. Knowl. Data Eng.* **35**(5), 4485–4498 (2023). <https://doi.org/10.1109/TKDE.2022.3153523>, <https://doi.org/10.1109/TKDE.2022.3153523>
7. Chen, X., Huo, H., Huan, J., Vitter, J.S.: An efficient algorithm for graph edit distance computation. *Knowl. Based Syst.* **163**, 762–775 (2019). <https://doi.org/10.1016/J.KNOSYS.2018.10.002>, <https://doi.org/10.1016/j.knosys.2018.10.002>
8. D’Ascenzo, A., Meffert, J., Mutzel, P., Rossi, F.: Enhancing graph edit distance computation: Stronger and orientation-based ILP formulations. *Proc. VLDB Endow.* **18**(11), 4737–4749 (2025), <https://www.vldb.org/pvldb/vol18/p4737-d%27ascenzo.pdf>
9. Gao, X., Xiao, B., Tao, D., Li, X.: A survey of graph edit distance. *Pattern Anal. Appl.* **13**(1), 113–129 (2010). <https://doi.org/10.1007/S10044-008-0141-Y>, <https://doi.org/10.1007/s10044-008-0141-y>
10. Gouda, K., Hassaan, M.: Csi_ged: An efficient approach for graph edit similarity computation. In: *32nd IEEE International Conference on Data Engineering, ICDE 2016, Helsinki, Finland, May 16–20, 2016*. pp. 265–276. IEEE Computer Society (2016). <https://doi.org/10.1109/ICDE.2016.7498246>, <https://doi.org/10.1109/ICDE.2016.7498246>
11. Justice, D., III, A.O.H.: A binary linear programming formulation of the graph edit distance. *IEEE Trans. Pattern Anal. Mach. Intell.* **28**(8), 1200–1214 (2006).

- <https://doi.org/10.1109/TPAMI.2006.152>, <https://doi.org/10.1109/TPAMI.2006.152>
12. Kim, J.: Efficient graph edit distance computation using isomorphic vertices. *Pattern Recognit. Lett.* **168**, 71–78 (2023). <https://doi.org/10.1016/J.PATREC.2023.03.002>, <https://doi.org/10.1016/j.patrec.2023.03.002>
 13. Lerouge, J., Abu-Aisheh, Z., Raveaux, R., Héroux, P., Adam, S.: New binary linear programming formulation to compute the graph edit distance. *Pattern Recognit.* **72**, 254–265 (2017). <https://doi.org/10.1016/J.PATCOG.2017.07.029>
 14. Lodi, A., Tramontani, A.: Performance variability in mixed-integer programming. In: *Theory driven by influential applications*, pp. 1–12. INFORMS (2013)
 15. Morris, C., Kriege, N.M., Bause, F., Kersting, K., Mutzel, P., Neumann, M.: TUDataset: A collection of benchmark datasets for learning with graphs. In: *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)* (2020), www.graphlearning.io
 16. Riesen, K.: *Structural Pattern Recognition with Graph Edit Distance - Approximation Algorithms and Applications*. *Advances in Computer Vision and Pattern Recognition*, Springer (2015). <https://doi.org/10.1007/978-3-319-27252-8>, <https://doi.org/10.1007/978-3-319-27252-8>
 17. Riesen, K., Fankhauser, S., Bunke, H.: Speeding up graph edit distance computation with a bipartite heuristic. In: Frascioni, P., Kersting, K., Tsuda, K. (eds.) *Mining and Learning with Graphs, MLG 2007, Firenze, Italy, August 1-3, 2007, Proceedings* (2007), http://mlg07.dsi.unifi.it/pdf/02_Riesen.pdf
 18. Xu, Z., Chen, P., Raveaux, R., Yang, X., Liu, H.: Deep graph matching meets mixed-integer linear programming: Relax or not ? *Pattern Recognition* **155**, 110697 (2024). <https://doi.org/https://doi.org/10.1016/j.patcog.2024.110697>, <https://www.sciencedirect.com/science/article/pii/S0031320324004485>